Software Architecture Document

Hostel Management System (HMS)

Team 1

Vaishnavi Goyal (PES2UG24CS571)

Yellala Sai Aneesh Reddy (PES2UG24CS615)

Vignesh Nunna (PES2UG24CS582)

Vishnu S R (PES2UG24CS595)

Version 1.1

Status: Final

12 November 2026

Revision history

Version	Date	Author	Change summary	Status
1.0	14-09-2026	Team 1	Initial SAD for the Hostel Management System: component diagram, two sequence diagrams, API design and security architecture, aligned to the Team 1 SRS	Draft
1.1	12-11-2026	Team 1	Final version after Sprint 2 and testing: related documents and open issues brought up to date; differences between this design and the built system are listed in DEVIATIONS.md (rows 1 to 18); architecture unchanged	Final

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			
Project Guide			
Hostel Warden (Client)			

Table of Contents

No.	Section
1	Introduction
2	Document overview
3	Architecture
4	Design
5	Appendices

1. Introduction

1.1 Purpose

This document is the Software Architecture Document (SAD) for the Hostel Management System (HMS). It specifies how the system realises the requirements set out in the Team 1 Software Requirements Specification (SRS): the components the system is built from, the technology chosen for each, the interfaces between them, the security architecture, and the detailed design of two representative end-to-end flows. Where the SRS states *what* the system must do, this document states *how* it is built to do it.

1.2 Scope

Covers the architecture of the HMS application described in the SRS: admission and allocation, fee and payment, gate and outpass, mess, complaints, visitors, and administration and reporting. It includes the component (UML) diagram, component responsibilities, the chosen architecture pattern, the technology stack, risk and security analysis, two UML sequence diagrams, the REST API design for three components, error-handling and monitoring conventions, and UX design notes.

Out of scope: the requirements themselves (see the SRS), the detailed test plan, and the internal design of third-party systems the HMS integrates with (the institutional ERP, the payment gateway, the SMS/email provider and the RFID/biometric reader), beyond the interface each exposes to the HMS.

1.3 Audience

Developers implementing the system, QA engineers writing test cases against the API design, security reviewers, instructors and assessment evaluators, and any future team member who needs to understand the system's structure before changing it.

1.4 Definitions

Term	Meaning
HMS	Hostel Management System
SRS	Software Requirements Specification (the companion document to this SAD)
RTM	Requirements Traceability Matrix, held in SRS section 8
REST	Representational State Transfer, the architectural style used for all internal APIs
RBAC	Role-Based Access Control
MFA	Multi-Factor Authentication
TLS	Transport Layer Security
PCI-DSS	Payment Card Industry Data Security Standard
SLA	Service Level Agreement
RTO / RPO	Recovery Time Objective / Recovery Point Objective
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege — the threat-modelling framework used in 3.9
WCAG	Web Content Accessibility Guidelines

2. Document Overview

2.1 How to use this document

Section 3 (Architecture) is written for anyone who needs the system's overall shape: the component diagram, why the architecture pattern was chosen, the technology stack, the risk register, and the security threat model. Section 4 (Design) goes one level deeper, into two concrete request flows and the REST contracts of three components, and is written for whoever implements or tests those flows. Section 5 collects the glossary, references and tooling used to produce this document.

Every architectural element that exists to satisfy a specific requirement names that requirement's ID (`HMS-F-###`, `HMS-NF-###`, `HMS-SR-###`, or `SO-#`) so it can be cross-checked against the SRS.

2.2 Related Documents

Document	Relationship
`SRS_Team1_Hostel_Management_System` (this team's SRS)	Source of all functional, non-functional and security requirements referenced here; section 8 holds the full Requirements Traceability Matrix.
`SAD_Template for SE Reference.docx` (instructor template)	The structure this document follows.
Software Test Plan (STP)	Version 1.1 (Final) is the test plan for this design; its security validation is section 5.1 and its test cases trace back to the SRS.
`DEVIATIONS.md` (project repository)	Every place where the built system differs from this document, with the reason.

3. Architecture

3.1 Goals & Constraints

Goals

- Allocation, fee and approval records are correct and auditable: every action is attributable to an actor and cannot be silently altered (SO-2, HMS-SR-004).
- Resident personal, guardian and medical data — some residents are minors — is confidential by default (SO-1, HMS-SR-001, HMS-SR-002).
- The system is available 99.5% of the time during the academic session (HMS-NF-002) and returns pages within 2 seconds for 95% of requests at 200 concurrent users (HMS-NF-001).
- Low-skill actors — the security guard at the gate, maintenance staff on a phone — can use their screens with minimal training (HMS-NF-004).

Constraints

- No card data is ever stored by the HMS; payment capture is delegated to a PCI-DSS compliant gateway (HMS-SR-005).
- Gender-segregated allocation is a hard invariant, not a preference (SRS 2.5); it must be enforced in the data-access layer, not only in the UI.
- The institutional ERP is the authoritative source of student identity; the HMS reads it but never writes to it (SRS 2.5).
- The gate terminal's network connection may be intermittent; entry/exit events must be buffered locally and replayed in order (SRS 2.5, 3.4).
- Resident, fee and gate-entry records are retained for 5 years, after which personal data is archived or anonymised (HMS-NF-005).
- The system is built and operated by a four-person student team, which favours an architecture with a low operational footprint over one that needs dedicated platform engineering.

3.2 Stakeholders & Concerns

Stakeholder	Primary concern
Resident	Privacy of personal data; a simple, fast mobile experience
Guardian	Reliable, timely notification of outpass and dues events
Warden	Fast allocation and approval workflow; dashboards over deep menus
Security guard	A gate screen that is fast and forgiving of low typing skill
Maintenance staff	A mobile-friendly complaint queue
Mess manager	Accurate opt-out tracking and consumption billing
Hostel administrator	Reporting, auditability, control of masters and users
Institution / regulators	PCI-DSS and data-protection compliance
Development & maintenance team	Modularity and testability without a heavy operational burden

3.3 Component (UML) Diagram

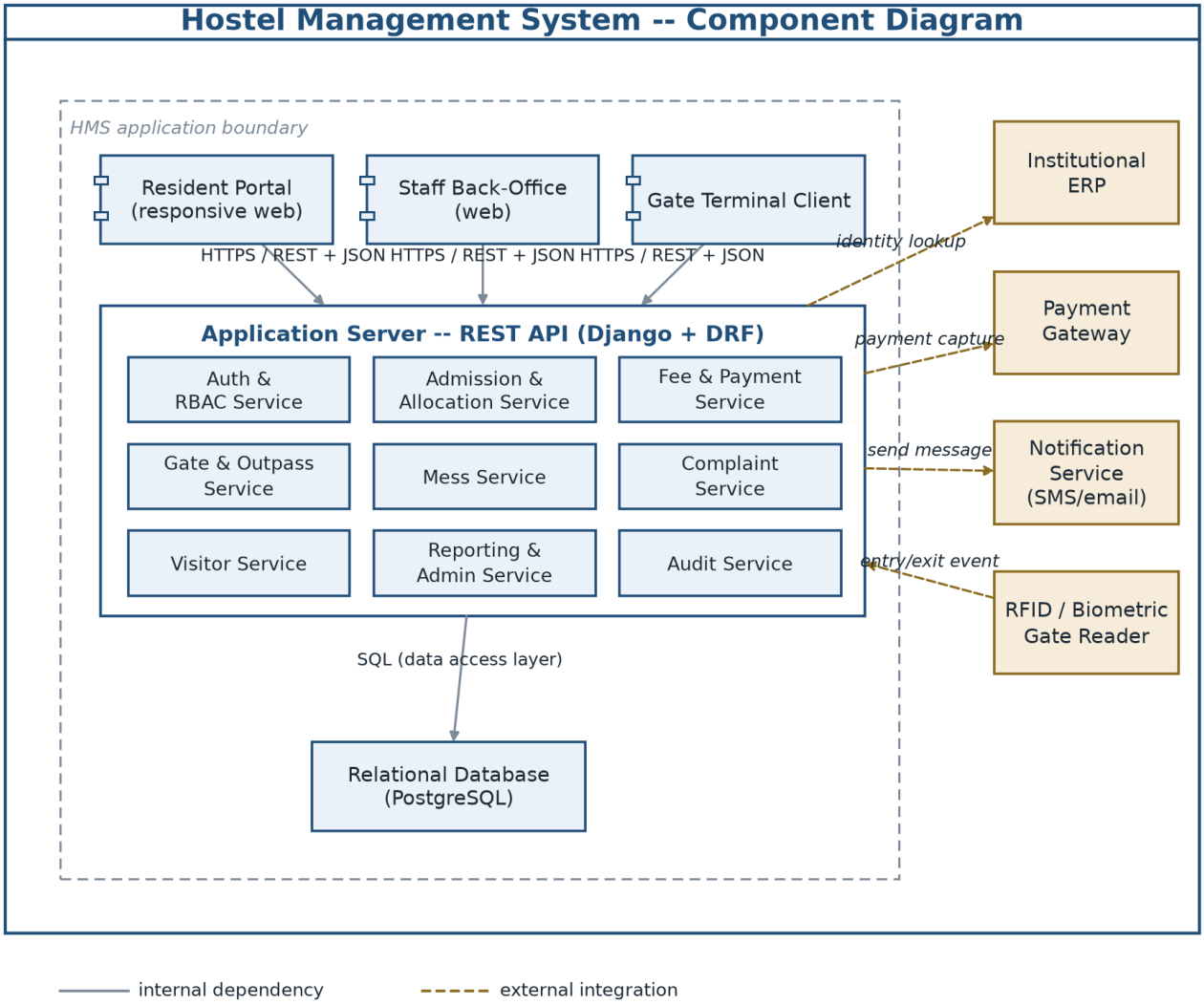


Figure 1 — Component diagram: presentation, application and data tiers, with the external systems the SRS names

3.4 Component Descriptions

- **Resident Portal** — responsive web client for residents: application, fee payment and receipts, outpass requests, mess opt-out and complaints.
- **Staff Back-Office** — web client for the warden, security guard, maintenance staff, mess manager and administrator, dashboard-led per SRS 2.3.
- **Gate Terminal Client** — the kiosk UI at the hostel gate; high-contrast, minimal typing, buffers entry/exit events during a network outage.
- **Auth & RBAC Service** — authentication, session management and role-based authorisation, with MFA for Warden and Administrator accounts (HMS-SR-002, HMS-SR-003).
- **Admission & Allocation Service** — application intake, eligibility checks and gender/block/room-type bed allocation (HMS-F-001 to HMS-F-003).
- **Fee & Payment Service** — invoicing, payment-gateway integration, receipting and overdue tracking (HMS-F-010 to HMS-F-013).
- **Gate & Outpass Service** — entry/exit capture, outpass workflow, curfew alerts and night attendance (HMS-F-020 to HMS-F-023).
- **Mess Service** — menu publication, opt-out and consumption-based billing (HMS-F-030, HMS-F-031).
- **Complaint Service** — complaint lodging, assignment, lifecycle and SLA escalation (HMS-F-040 to HMS-F-042).
- **Visitor Service** — visitor registration and visiting-hours enforcement (HMS-F-050).
- **Reporting & Admin Service** — occupancy/dues/complaint/mess reports, room-inventory and fee-structure masters, and the vacating/no-dues workflow (HMS-F-060 to HMS-F-062).
- **Audit Service** — a tamper-evident, append-only log of every allocation, fee, outpass, override and account-management action (HMS-SR-004).
- **Relational Database (PostgreSQL)** — the single system of record, reachable only through each service's own data-access code.

3.5 Chosen Architecture Pattern and Rationale

Chosen pattern: layered (n-tier) architecture, implemented as a modular monolith. The application server in Figure 1 is a single deployable Django process, internally divided into the eleven service modules of 3.4, each its own Django app with a REST controller group (via Django REST Framework), above one relational database.

Rationale. The SRS already fixes a three-tier shape — browser client, REST application server, relational database (SRS 2.1) — so this document chooses how to implement that tier, not whether to have it. A modular monolith keeps a transaction like bed allocation (HMS-F-002) or the no-dues/refund calculation on vacating (HMS-F-062), both of which touch more than one module, inside a single ACID database transaction; splitting those modules into independently deployed services would require distributed transactions or a saga to get the same guarantee. At the project's stated scale — 2,000 resident records, 200 concurrent users (HMS-NF-001) — that complexity buys nothing. A monolith is also operable by a four-person team without a service mesh, container orchestration platform or distributed tracing stack, and Django's app-per-module layout gives that structure for free rather than something the team has to impose by convention — a fit that also matches the team's existing Python background (3.6).

Rejected: microservices per module. Rejected for the reasons above — the added operational and transactional complexity is disproportionate to the scale, and the team does not have the platform-engineering capacity to run it.

Rejected: a server-rendered monolith with no REST layer. Rejected because SRS 2.1 and 3.3 already fix REST/JSON as the client-server contract, and a documented API (section 4.3) is required for the resident portal, staff back-office and gate terminal to be built and tested independently of the backend team.

3.6 Technology Stack & Data Stores

Chosen for a team building in Python: Django keeps the whole backend in one language the team already knows, and its app-per-module layout is a natural fit for the modular monolith of 3.5.

Layer	Choice	Notes
Backend	Python 3.12, Django + Django REST Framework (DRF)	One process per 3.5; each service module of 3.4 is a Django app; Django's built-in admin gives the Hostel Administrator screens a head start
Frontend (portal / back-office)	Django templates with Bootstrap 5, server-rendered	No separate frontend language for the team to learn; pages built to WCAG 2.1 AA (HMS-NF-004)
Gate terminal	A small server-rendered page plus vanilla JS, with a local IndexedDB buffer	Buffers scans during a network outage, replays them in order on reconnect via the DRF API (SRS 2.5, 3.4)
Database	PostgreSQL, via the Django ORM and Django's built-in migrations	Matches SRS 2.4; single relational store
Reporting	WeasyPrint or ReportLab for PDF, openpyxl for XLSX	Receipts (HMS-F-012), allocation letters, and the reports of HMS-F-060
Identity integration	`requests`-based HTTPS/JSON client against the institutional ERP, read-only	Isolated behind an adapter inside Admission & Allocation Service (3.7)
Payment integration	HTTPS client against a PCI-DSS compliant gateway, signed webhook callback	No cardholder data is persisted (HMS-SR-005)
Notifications	Templated SMS/email client, retried with a dead-letter queue after 3 attempts	Matches SRS 3.4
Security	TLS 1.2+ everywhere; Django's Argon2 password hasher; TOTP-based MFA (`django-otp`) for Warden/Administrator; session-based auth with a 15-minute idle expiry, DRF token/JWT for the gate terminal and webhooks	HMS-SR-001, HMS-SR-002, HMS-SR-003
Scheduled jobs	Celery with a Redis broker (or Django management commands under cron for a simpler start) for overdue-invoice flags, curfew checks and SLA escalation	HMS-F-013, HMS-F-022, HMS-F-042
Operations	Nightly automated `pg_dump` backup	RPO 24h, RTO 4h (HMS-NF-003)

3.7 Risks & Mitigations

Risk	Impact	Mitigation
Gate terminal network outage	Entry/exit events lost	Local buffering with ordered replay on reconnect (SRS 3.4)
Payment gateway downtime or timeout	Fee payments stuck mid-flow	Retry with exponential backoff and a dead-letter queue; invoice stays Unpaid rather than being marked Paid incorrectly (HMS-F-011)
ERP API or contract change	Admission eligibility lookup breaks	Integration isolated behind an adapter in Admission & Allocation Service
Peak-season load at start of term	Allocation/fee pages slow beyond target	Connection pooling, indexed queries, load-tested against HMS-NF-001 before rollout
Resident PII or medical-data breach	Regulatory and reputational harm; minors affected	TLS in transit, encryption of sensitive columns at rest, least-privilege RBAC, full audit trail (HMS-SR-001 to HMS-SR-004)
Single relational database is a single point of failure	Data loss or extended outage	Nightly backup and a documented quarterly restore drill, RTO 4h / RPO 24h (HMS-NF-003)
RFID/biometric vendor lock-in	Hardware change forces rework	Vendor event API isolated behind a device adapter in Gate & Outpass Service (SRS 2.5)
Incorrect gender/block allocation	Violates a hard institutional rule	Gender/block filter enforced in the data-access query, not only in the UI (HMS-F-002)

3.8 Traceability to Requirements

The table below maps each requirement group to the component that realises it; full requirement-to-test-case traceability is in SRS section 8.

Requirement(s)	Component
HMS-F-001 – HMS-F-003	Admission & Allocation Service
HMS-F-010 – HMS-F-013	Fee & Payment Service
HMS-F-020 – HMS-F-023	Gate & Outpass Service
HMS-F-030, HMS-F-031	Mess Service
HMS-F-040 – HMS-F-042	Complaint Service
HMS-F-050	Visitor Service
HMS-F-060 – HMS-F-062	Reporting & Admin Service
HMS-NF-001	Resident Portal, Staff Back-Office, Application Server (performance budget spans the REST layer)
HMS-NF-002, HMS-NF-003, HMS-NF-005	Relational Database / operations (backup, retention, archival jobs)
HMS-NF-004	Resident Portal (accessibility and mobile layout)
HMS-SR-001	All tiers (TLS on every connection)
HMS-SR-002, HMS-SR-003	Auth & RBAC Service
HMS-SR-004	Audit Service
HMS-SR-005	Fee & Payment Service, Payment Gateway
HMS-SR-006	All modules (input validation at the REST boundary)

3.9 Security Architecture

Threat model (STRIDE).

Threat	Example in the HMS	Mitigation	Requirement
Spoofing	A forged resident or staff login	MFA for Warden/Administrator, salted password hashes, signed session tokens	HMS-SR-003
Tampering	Altering an allocation or fee record after the fact	Tamper-evident, append-only audit log; no application path edits it	HMS-SR-004
Repudiation	A warden denies having approved a waiver or allocation	Every privileged action logs actor, action, timestamp and outcome	HMS-SR-004, SO-2
Information disclosure	Resident medical or guardian details exposed to another resident	RBAC least privilege, TLS in transit, encryption at rest for identity-proof data	HMS-SR-001, HMS-SR-002, SO-1
Denial of service	Portal or gate terminal flooded near an admission or curfew deadline	Rate limiting, validated input, capacity tested to HMS-NF-001	HMS-NF-001, HMS-SR-006
Elevation of privilege	A resident account calling a warden-only allocation endpoint	RBAC enforced server-side on every endpoint, tested per role	HMS-SR-002

4. Design

4.1 Design Overview

Each service module in 3.4 is one Django app with its own models, DRF serializers and views; cross-module calls within the application server are ordinary in-process method calls unless the sequence diagrams below mark them as a network call to an external system. Where a single business action spans modules — bed allocation writing to both Admission & Allocation and Audit, or vacating writing to Fee, Mess and Reporting & Admin — the write happens inside one database transaction, so the modular-monolith choice in 3.5 holds in the data layer as well as the package layer.

4.2 UML Sequence Diagrams

Two sequence diagrams are provided, each covering a flow that spans several components and an external system. Figure 2 covers online fee payment (HMS-F-011 to HMS-F-013). Figure 3 covers the outpass workflow, including the guardian notifications required at both approval and return (HMS-F-021).

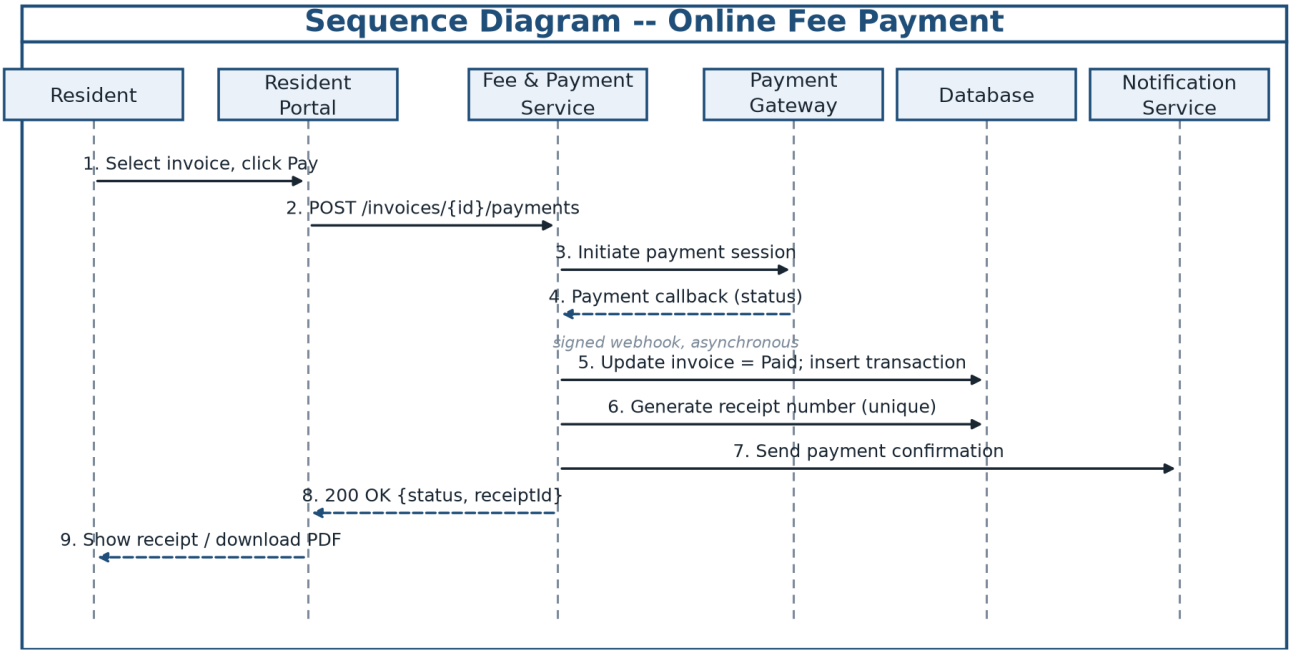


Figure 2 — Sequence diagram: online fee payment

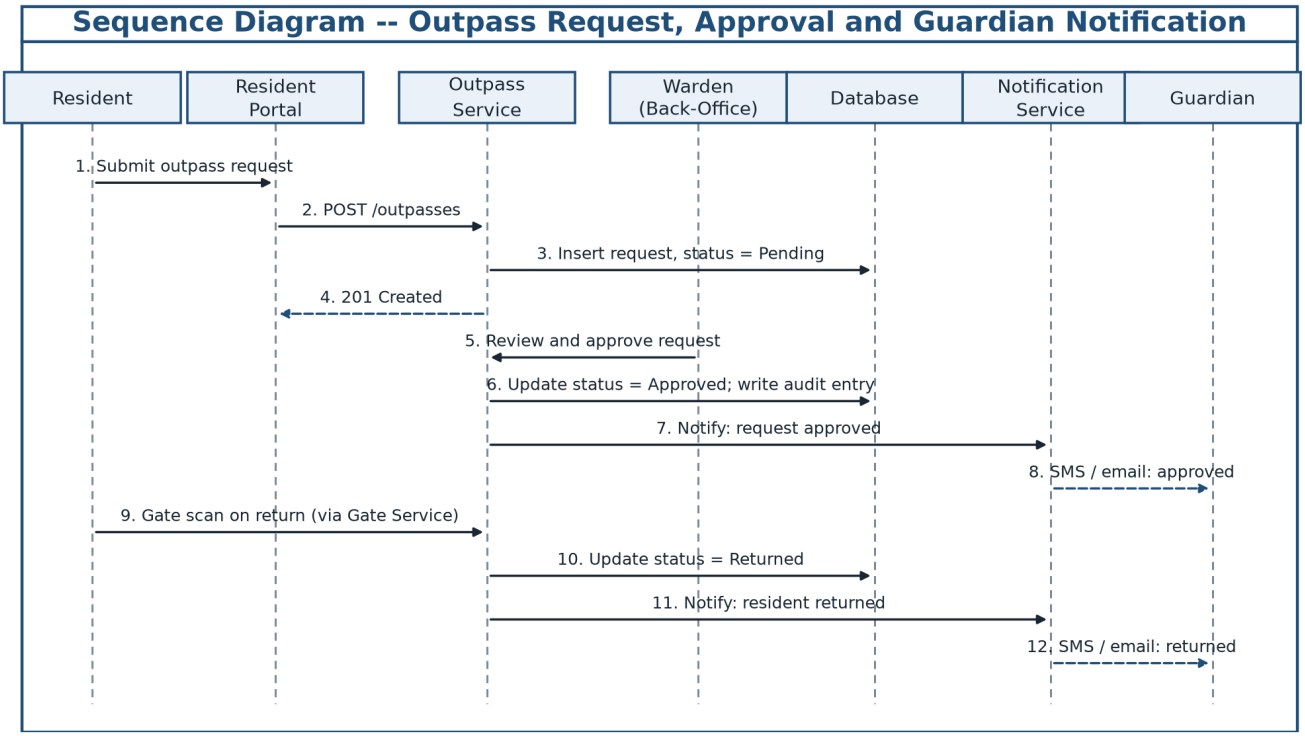


Figure 3 — Sequence diagram: outpass request, approval and guardian notification

4.3 API Design

REST contracts for three components: Admission & Allocation Service, Fee & Payment Service, and Gate & Outpass Service. All endpoints require an authenticated session (Auth & RBAC Service) except where noted, are served over HTTPS, and return the error envelope defined in 4.4.

Admission & Allocation Service

Endpoint: `POST /api/v1/applications`
Request: `{enrolmentNumber, personalDetails, guardianDetails, medicalDetails, preferredBlock, roomType}`
Response: `{applicationNumber, status: "Submitted"}`
Errors: `400` validation failed · `409` duplicate application for this enrolment number and session (HMS-F-001)

Endpoint: `POST /api/v1/allocations`
Request: `{applicationNumber}`
Response: `{bedId, block, room, allocatedAt}`
Errors: `404` application not found or not approved · `409` no vacant bed matches the applicant's gender, block and room type (HMS-F-002)

Fee & Payment Service

Endpoint: `POST /api/v1/invoices/{invoiceId}/payments`
Request: `{method, returnUrl}`
Response: `{paymentSessionId, redirectUrl}`
Errors: `404` invoice not found · `409` invoice already paid · `502` gateway unavailable (HMS-F-011)

Endpoint: `POST /api/v1/payments/callback` (gateway webhook, unauthenticated but signature-verified)
Request: `{paymentSessionId, status, gatewayRef, signature}`
Response: `{received: true}`
Errors: `400` invalid signature · `404` unknown payment session

Gate & Outpass Service

Endpoint: `POST /api/v1/outpasses`
Request: `{residentId, destination, contactNumber, expectedReturnAt}`
Response: `{outpassId, status: "Pending"}`
Errors: `400` validation failed (HMS-F-021)

Endpoint: `PATCH /api/v1/outpasses/{id}/decision`
Request: `{decision: "Approved" | "Rejected", wardenId, reason}`
Response: `{outpassId, status}`
Errors: `403` caller is not a warden · `404` not found · `409` already decided

4.4 Error Handling, Logging & Monitoring

Errors. Every API error returns a standard envelope — `{timestamp, path, status, error, message, traceId}` — and never includes a stack trace or resident personal data in the body.

Logging. Structured logs carry a correlation/trace ID per request. Resident medical details, guardian contact details and identity-proof numbers are never written to logs (SO-1); the audit log (HMS-SR-004) is the only place a privileged action's actor and outcome are recorded.

Monitoring. Dashboards track: payment failure rate (HMS-F-011), curfew alert delivery latency (HMS-F-022), open SLA-breach count (HMS-F-042), API 95th-percentile latency (HMS-NF-001), and uptime (HMS-NF-002). Alerts fire when payment failure rate, API error rate, or SLA-breach count crosses a configured threshold, or when the nightly backup job fails (HMS-NF-003).

4.5 UX Design

Resident Portal: mobile-first, large touch targets, WCAG 2.1 AA, and the smallest number of steps to pay a fee, request an outpass or lodge a complaint (HMS-NF-004).

Staff Back-Office: dashboard-first — pending approvals, curfew alerts and the SLA-breach queue are the landing view, not a menu tree, matching the moderate computer skill and daily-use pattern of the warden (SRS 2.3).

Gate Terminal: a single high-contrast screen showing the scanned resident's photo, name, room and current IN/OUT state, operable with minimal typing and an optional buzzer for a violation alert (SRS 3.1, 3.2).

4.6 Open Issues & Next Steps

Status after Sprint 2 and testing (12 November 2026):

- **MFA method: decided.** Warden and Administrator accounts use authenticator-app TOTP (django-otp), built in Sprint 1. SMS one-time codes were not used.
- **Disaster-recovery site: still open.** Nothing beyond the nightly backup is designed; the restore drill met the RPO and RTO (System Validation Report, TC-OPS-01).
- **Replay policy for gate events: resolved in the API.** Each event keeps its own time and a repeated event is ignored, so a client that buffers events during an outage can send them later in any order. The browser-side buffer itself was not built (DEVIATIONS 15).
- **Load test against HMS-NF-001: run, with a caveat.** 200 concurrent users and 2,000 seeded residents gave a 95th percentile of 1.6 s with no failures, on a developer laptop rather than a production-like server (DEVIATIONS 14).
- **Future enhancements:** biometric MFA at the gate terminal and QR-based visitor pre-registration (mirrors the SRS's own open items).

5. Appendices

5.1 Glossary

HMS, Resident, Warden, Outpass, Curfew, No-dues, Bed, RFID, SLA, RTM, RTO / RPO, MFA, RBAC, REST, STRIDE, WCAG — see 1.4 for definitions; SRS 1.4 carries the full domain glossary (Resident, Warden, Outpass, Curfew, No-dues, Bed).

5.2 References

IEEE 42010 (architecture description) · OWASP ASVS / OWASP Top 10 · NIST SP 800-160 · PCI-DSS · WCAG 2.1 · `SRS_Team1_Hostel_Management_System` (this team's SRS).

5.3 Tools

matplotlib (component and sequence diagrams, `diagrams/make_diagrams.py`) · pandoc with a LaTeX and `--reference-doc` pipeline (`build.sh`, matching the SRS build) · an OpenAPI/Postman-style specification for the API contracts of 4.3.